

Accenture JavaScript Assessment-Ready Complete Notes

Theory + Practical DOM Patterns + Assessment Answer Recipes

Long-Term Revision Edition

2026

Contents

Scope and purpose	5
What is intentionally not covered in depth	5
Quick Navigation	5
1. JavaScript in an HTML/CSS Page	6
1.1 What role does JavaScript play?	6
Why each JavaScript line exists	6
Core frontend mental model	6
1.2 Connecting a JavaScript file	7
Why <code>defer</code> is useful	7
Assessment trap	7
2. Variables, Types, Coercion, and Operators	7
2.1 <code>var</code> , <code>let</code> , and <code>const</code>	7
<code>let</code>	7
<code>const</code>	7
<code>const</code> object/array mutation trap	8
Why this matters in DOM code	8
2.2 Block scope	8
2.3 Important primitive values and <code>typeof</code>	9
2.4 <code>+</code> and coercion	9
Left-to-right evaluation trap	9
2.5 <code>==</code> versus <code>===</code>	9
2.6 Truthy and falsy	10
2.7 Increment operators	10
3. Conditions and Logical Operators	11
3.1 <code>if</code> , <code>else if</code> , and <code>else</code>	11
Ordering trap	11
3.2 Independent <code>if</code> statements versus an <code>else if</code> chain	11
3.3 Logical AND <code>&&</code>	12
Practical validation use	12
3.4 Logical OR <code> </code>	12
3.5 Logical NOT <code>!</code> and double NOT <code>!!</code>	12
Practical checkbox use	13
3.6 Logical operator precedence	13
3.7 Short-circuit evaluation	13
3.8 Ternary operator	14
4. Loops and Execution Tracing	14
4.1 <code>for</code> loop execution order	14
4.2 Off-by-one trap	14
4.3 <code>while</code>	14
4.4 <code>do...while</code>	15
4.5 <code>break</code>	15

4.6	<code>continue</code>	16
4.7	Accumulator and counter patterns	16
4.8	Combined trace pattern	16
5.	Functions, Scope, Return, and Hoisting	17
5.1	Function declaration	17
5.2	<code>return</code> has two jobs	17
5.3	<code>console.log()</code> is not <code>return</code>	18
5.4	Missing and extra arguments	18
5.5	Default parameters	18
5.6	Local and global scope	18
	Why this matters in function questions	19
5.7	Function declaration hoisting	19
5.8	Arrow functions	19
	Critical arrow return trap	20
5.9	Callback functions in event code	20
6.	Small Array Patterns Needed for Assessment Tasks	20
6.1	<code>push()</code> return value	21
6.2	Reference mutation	21
6.3	<code>map()</code> and the missing return trap	21
6.4	<code>filter()</code> plus <code>map()</code> chaining	21
	Why this appears again in DOM work	22
7.	DOM Fundamentals	22
7.1	What is the DOM?	22
7.2	<code>document</code>	22
8.	Selecting Elements	22
8.1	<code>getElementById()</code>	22
8.2	<code>querySelector()</code>	23
8.3	<code>querySelectorAll()</code>	23
	Major trap	23
	Why <code>forEach</code> is used here	24
8.4	Selection decision table	24
9.	Reading and Modifying Page Content	24
9.1	<code>textContent</code>	24
9.2	<code>.value</code>	24
	Assessment distinction	25
9.3	<code>.checked</code>	25
9.4	<code>.dataset</code>	25
9.5	<code>textContent</code> versus <code>innerHTML</code>	25
10.	Classes, Styles, and Attributes	26
10.1	<code>classList.add()</code>	26
10.2	<code>classList.remove()</code>	26
10.3	<code>classList.toggle()</code>	26
10.4	<code>classList.contains()</code>	26
10.5	Direct style modification	26
10.6	Attributes	27
	Assessment question pattern	27
11.	Events	27
11.1	What is an event?	27
11.2	<code>addEventListener()</code>	27
11.3	<code>click</code>	28
11.4	<code>input</code>	28
11.5	<code>change</code>	28
11.6	<code>submit</code>	28
11.7	<code>event.target</code>	28
11.8	<code>event.currentTarget</code>	28
12.	Forms and Validation	29

12.1 The basic form flow	29
Why each line exists	29
12.2 Why <code>.trim()</code> matters	29
12.3 Checkbox validation	30
12.4 Multiple checked checkboxes	30
12.5 Radio buttons	30
Null trap	31
12.6 Select element	31
12.7 Error messages in the page	31
12.8 Invalid input styling	32
12.9 Early-return validation pattern	32
12.10 <code>required</code> , <code>readonly</code> , and <code>disabled</code>	32
12.11 Button types inside forms	33
13. Creating, Inserting, and Removing Elements	33
13.1 <code>createElement()</code>	33
Why each concept reappears	33
13.2 <code>append()</code> versus <code>prepend()</code>	34
13.3 Add-item assessment pattern	34
13.4 <code>remove()</code>	34
13.5 Dynamic delete button	35
14. DOM Traversal	35
14.1 <code>parentElement</code>	35
Trap	35
14.2 <code>children</code>	35
14.3 <code>nextElementSibling</code> and <code>previousElementSibling</code>	36
14.4 <code>closest()</code>	36
15. Event Bubbling and Delegation	36
15.1 Event bubbling	36
15.2 Event delegation	37
Why each line exists	37
Why delegation helps with dynamic elements	37
15.3 <code>matches()</code>	37
15.4 <code>preventDefault()</code> versus <code>stopPropagation()</code>	38
16. NexaCorp Practical Reference Patterns	38
16.1 Preview profile name	38
Why previously learned concepts are reused	38
Assessment prompt variation	39
16.2 Toggle announcements	39
16.3 Make only one course active	39
16.4 Read selected skills	39
16.5 Form submit validation	40
17. Assessment Question Patterns and Answer Recipes	41
Pattern 1: Exact output prediction	41
Prompt style	41
Answer method	41
Pattern 2: Return-value trap	41
Prompt style	41
Answer	41
Pattern 3: Missing <code>return</code>	41
Prompt style	41
Diagnosis	42
Minimum fix	42
Pattern 4: Select one DOM element	42
Requirement	42
Pattern 5: Select all elements	42
Requirement	42
Pattern 6: Read input and change text	42
Requirement	42

Pattern 7: Checkbox state	43
Requirement	43
Pattern 8: All selected checkboxes	43
Pattern 9: Selected radio	43
Pattern 10: Hide/show using CSS	43
Pattern 11: Prevent form submission and validate	43
Pattern 12: Create and append an element	44
Pattern 13: Remove the clicked card	44
Pattern 14: Change an attribute	44
Pattern 15: Bug caused by HTML button type	44
18. High-Frequency Traps	44
19. Mixed Assessment Practice with Answers	45
Q1 - Scope and function parameter	45
Answer	45
Q2 - Independent conditions	45
Answer	46
Q3 - Function control flow	46
Answer	46
Q4 - DOM selector fill-in	46
Answer	46
Q5 - Current input value	46
Answer	47
Q6 - NodeList bug	47
Fix	47
Q7 - Toggle visibility	47
Answer	47
Q8 - Form submit	47
Answer	47
Q9 - Whitespace validation	47
Answer	48
Q10 - Selected checkboxes	48
Answer	48
Q11 - Radio null trap	48
Answer	48
Q12 - Dynamic list item	48
Answer pattern	48
Q13 - Delete clicked card using delegation	49
Answer pattern	49
Q14 - <code>target</code> versus <code>currentTarget</code>	49
Answer	49
Q15 - <code>preventDefault</code> versus propagation	49
Answer	49
Q16 - Full integrated implementation	49
Answer	49
Why every line exists	50
20. Last-Minute Revision Sheet	50
Core JavaScript	50
DOM selection	50
DOM values	50
Events	51
Forms	51
Classes	51
Attributes	51
Dynamic DOM	51
Traversal	51
Delegation pattern	52
Universal assessment workflow	52
21. Coverage Checklist	52
Core language	52

Functions	52
Small JavaScript array support	53
DOM	53
CSS control from JavaScript	53
Events	53
Forms	53
Dynamic DOM	54
Traversal and delegation	54

Final Recall Map 54

A long-term revision guide for HTML/CSS/JavaScript assessment tasks.

Focus: JavaScript concepts already covered in the preparation track, especially the concepts most useful when JavaScript is implemented with HTML and CSS.

Scope and purpose

These notes are designed to remain useful even after a long gap. They do not assume that you remember the surrounding conversation or the original project. Each important concept is explained using the same repeatable structure:

Problem -> Why the concept is needed -> Syntax -> Execution -> Practical example -> Common trap -> Assessment-style question and answer pattern.

The emphasis is not on deep JavaScript internals. The emphasis is on being able to read unfamiliar code, predict output, identify bugs, fill missing lines, and implement small browser features inside an HTML/CSS page.

This is **not an official Accenture syllabus**. It is an assessment-oriented study guide built around the JavaScript and frontend patterns covered during preparation.

What is intentionally not covered in depth

- Advanced prototypes and prototype chains
- Deep closure internals
- Complex asynchronous JavaScript
- Framework-specific React concepts
- Advanced regular expressions
- Complex object-oriented JavaScript patterns
- Large algorithmic array/string problems already covered through DSA preparation

When a small array or string operation is required by a DOM task, the required JavaScript-specific behavior is explained at the point of use.

Quick Navigation

1. JavaScript in an HTML/CSS Page
2. Variables, Types, Coercion, and Operators
3. Conditions and Logical Operators
4. Loops and Execution Tracing
5. Functions, Scope, Return, and Hoisting
6. Small Array Patterns Needed for Assessment Tasks
7. DOM Fundamentals
8. Selecting Elements
9. Reading and Modifying Page Content
10. Classes, Styles, and Attributes
11. Events
12. Forms and Validation
13. Creating, Inserting, and Removing Elements
14. DOM Traversal
15. Event Bubbling and Delegation

16. NexaCorp Practical Reference Patterns
 17. Assessment Question Patterns and Answer Recipes
 18. High-Frequency Traps
 19. Mixed Assessment Practice with Answers
 20. Last-Minute Revision Sheet
 21. Coverage Checklist
-

1. JavaScript in an HTML/CSS Page

1.1 What role does JavaScript play?

HTML provides **structure**, CSS provides **presentation**, and JavaScript provides **behavior**.

Example:

```
<h2 id="status">Pending</h2>
<button id="complete-btn" type="button">Complete</button>
```

CSS can decide how the status looks:

```
.completed {
  color: green;
  font-weight: bold;
}
```

JavaScript can make the page react to a click:

```
const status = document.getElementById("status");
const button = document.getElementById("complete-btn");

button.addEventListener("click", function () {
  status.textContent = "Completed";
  status.classList.add("completed");
});
```

Why each JavaScript line exists

```
const status = document.getElementById("status");
```

The browser page may contain many elements. JavaScript first needs a reference to the exact element that must be modified.

```
const button = document.getElementById("complete-btn");
```

The button is selected because the behavior must happen **when this button is clicked**.

```
button.addEventListener("click", function () {
```

The callback is registered for a future click. The callback does not run immediately during page load.

```
status.textContent = "Completed";
```

`textContent` changes the text stored in the selected DOM element.

```
status.classList.add("completed");
```

JavaScript changes state by adding a CSS class. CSS remains responsible for the visual appearance.

Core frontend mental model

```
SELECT
-> LISTEN
-> READ
-> VALIDATE
-> MODIFY
```

This pattern solves a large percentage of small HTML/CSS/JavaScript assessment tasks.

1.2 Connecting a JavaScript file

A clean approach is:

```
<head>
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
```

Why defer is useful

Without careful script placement, JavaScript can execute before the required HTML elements have been parsed. Then a lookup can return `null`.

```
const button = document.getElementById("preview-btn");
```

If the button does not yet exist in the DOM when this line runs, `button` is `null`, and the next line can fail:

```
button.addEventListener("click", function () {});
```

Using `defer` allows HTML parsing to finish before the script executes.

Another simple approach is placing the script just before `</body>`:

```
<script src="script.js"></script>
</body>
```

Assessment trap

If you see:

```
const btn = document.getElementById("save-btn");
btn.addEventListener("click", handler);
```

and the error says that `addEventListener` cannot be read from `null`, check whether:

- the ID is wrong,
- the script ran before the element existed,
- or the element is not present in the HTML.

2. Variables, Types, Coercion, and Operators

2.1 var, let, and const

```
var    -> function-scoped, reassignment allowed
let    -> block-scoped, reassignment allowed
const  -> block-scoped, reassignment not allowed
```

let

```
let x = 10;
x = 20;
```

```
console.log(x);
```

Output:

20

const

```
const x = 10;
x = 20;
```

Result:

TypeError: Assignment to constant variable

The variable binding cannot be assigned a new value.

const object/array mutation trap

```
const user = {  
  name: "Aman"  
};
```

```
user.name = "Suyash";
```

```
console.log(user.name);
```

Output:

Suyash

Why? `const` prevents reassignment of the variable reference. It does not automatically freeze the object.

Invalid:

```
const user = { name: "Aman" };
```

```
user = { name: "Ravi" };
```

The variable is being assigned a new object reference.

Why this matters in DOM code

You frequently write:

```
const button = document.getElementById("save-btn");
```

The variable should continue referring to the same DOM element. The DOM element itself can still change:

```
button.textContent = "Saved";  
button.classList.add("active");
```

`const` does not make the DOM element immutable.

2.2 Block scope

```
if (true) {  
  let x = 10;  
}
```

```
console.log(x);
```

Result:

ReferenceError

`let` is block-scoped.

Compare:

```
if (true) {  
  var x = 10;  
}
```

```
console.log(x);
```

Output:

10

`var` is not block-scoped.

2.3 Important primitive values and typeof

```
console.log(typeof "Hello");  
console.log(typeof 10);  
console.log(typeof true);  
console.log(typeof undefined);  
console.log(typeof null);
```

Output:

```
string  
number  
boolean  
undefined  
object
```

`typeof null` returning "object" is a historical JavaScript behavior and a common output question.

Another important trap:

```
console.log(Number("hello"));  
console.log(typeof NaN);
```

Output:

```
NaN  
number
```

NaN means "Not-a-Number", but its JavaScript type is still `number`.

2.4 + and coercion

```
console.log("10" + 5);  
console.log("10" - 5);
```

Output:

```
105  
5
```

With `+`, a string operand can cause concatenation. Arithmetic operators such as `-` attempt numeric conversion.

Left-to-right evaluation trap

```
console.log(10 + 20 + "30");
```

Execution:

```
10 + 20      -> 30  
30 + "30"    -> "3030"
```

Output:

```
3030
```

Compare:

```
console.log("10" + 20 + 30);
```

Execution:

```
"10" + 20    -> "1020"  
"1020" + 30  -> "102030"
```

Output:

```
102030
```

2.5 == versus ===

```
console.log(5 == "5");  
console.log(5 === "5");
```

Output:

```
true
false
```

- == allows type coercion.
- === compares without that coercive equality behavior.

Assessment preference: when writing your own normal comparisons, prefer strict equality unless the task specifically needs coercion.

Combined trap:

```
let value = "10";

if (value == 10 && value !== 10) {
  console.log("A");
} else {
  console.log("B");
}
```

Execution:

```
value == 10    -> true
value !== 10   -> true
true && true   -> true
```

Output:

A

2.6 Truthy and falsy

Important falsy values:

```
false
0
-0
0n
""
null
undefined
NaN
```

Examples:

```
0          -> falsy
"0"        -> truthy
""         -> falsy
"Hello"    -> truthy
null       -> falsy
undefined -> falsy
```

This matters directly in DOM validation.

```
const selectedGender = document.querySelector('input[name="gender"]:checked');

if (!selectedGender) {
  console.log("Select gender");
}
```

If no matching radio exists, `querySelector` returns `null`. `null` is falsy, so `!selectedGender` becomes `true`.

2.7 Increment operators

```
let x = 5;

console.log(x++);
```

```
console.log(x);  
console.log(++x);
```

Output:

```
5  
6  
7
```

- `x++`: use current value first, then increment.
- `++x`: increment first, then use the new value.

This often appears inside loop tracing questions.

3. Conditions and Logical Operators

3.1 if, else if, and else

```
let marks = 75;  
  
if (marks >= 90) {  
    console.log("A");  
} else if (marks >= 75) {  
    console.log("B");  
} else if (marks >= 50) {  
    console.log("C");  
} else {  
    console.log("Fail");  
}
```

Output:

B

An if / else if / else chain is checked top to bottom. The first true branch wins and the remaining branches are skipped.

Ordering trap

```
let marks = 95;  
  
if (marks >= 50) {  
    console.log("Pass");  
} else if (marks >= 90) {  
    console.log("Excellent");  
}
```

Output:

Pass

The more general condition was placed first, so the more specific condition is never reached.

3.2 Independent if statements versus an else if chain

```
let x = 10;  
  
if (x > 5) {  
    console.log("A");  
}  
  
if (x > 8) {  
    console.log("B");  
}
```

Output:

A
B

Both conditions are independently evaluated.

Compare:

```
let x = 10;

if (x > 5) {
  console.log("A");
} else if (x > 8) {
  console.log("B");
}
```

Output:

A

3.3 Logical AND &&

```
let age = 20;
let hasID = true;

if (age >= 18 && hasID) {
  console.log("Allowed");
}
```

Both conditions must be truthy.

```
true && true -> true
true && false -> false
false && true -> false
false && false -> false
```

Practical validation use

```
if (name !== "" && email !== "") {
  console.log("Basic fields present");
}
```

3.4 Logical OR ||

```
let isAdmin = false;
let isManager = true;

if (isAdmin || isManager) {
  console.log("Access");
}
```

Output:

Access

At least one operand must be truthy.

3.5 Logical NOT ! and double NOT !!

```
console.log(!true);
console.log(!0);
console.log(!"Hello");
```

Output:

```
false
true
false
```

!!value converts a value into its Boolean truthiness:

```
console.log(!!"Hello");
console.log(!!0);
console.log(!!"0");
```

Output:

```
true
false
true
```

Practical checkbox use

```
if (!terms.checked) {
  console.log("Accept terms");
}
```

terms.checked is a Boolean. If it is false, !false becomes true, so the error branch executes.

3.6 Logical operator precedence

Useful order:

```
!
&&
||
```

Example:

```
console.log(true || false && false);
```

&& is evaluated first:

```
false && false -> false
true || false -> true
```

Output:

```
true
```

Parentheses override the normal grouping:

```
console.log((true || false) && false);
```

Output:

```
false
```

3.7 Short-circuit evaluation

```
false && console.log("Hello");
```

No output. Once && sees a falsy left operand, the overall result cannot become truthy, so the right side is skipped.

```
true || console.log("Hello");
```

No output. Once || sees a truthy left operand, the right side is unnecessary.

A useful mental rule:

A && B -> first falsy value, otherwise last value

A || B -> first truthy value, otherwise last value

Examples:

```
console.log("Hello" && 10);    // 10
console.log(0 && "JS");        // 0
console.log("" || "Guest");    // Guest
console.log("Suyash" || "Guest"); // Suyash
```

3.8 Ternary operator

```
const result = age >= 18 ? "Adult" : "Minor";
```

Use ternary for a short two-way value decision. Avoid deeply nested ternaries in assessment code unless the prompt specifically expects them.

4. Loops and Execution Tracing

4.1 for loop execution order

```
for (let i = 1; i <= 4; i++) {  
  console.log(i);  
}
```

Execution order:

```
initialization  
-> condition  
-> body  
-> update  
-> condition  
-> body  
-> update  
...
```

Output:

```
1  
2  
3  
4
```

4.2 Off-by-one trap

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Output:

```
0  
1  
2  
3  
4
```

At $i = 5$, $5 < 5$ is false.

This same rule matters when iterating over DOM collections or arrays:

```
for (let i = 0; i < items.length; i++) {  
  // valid indices  
}
```

4.3 while

```
let i = 1;  
  
while (i <= 3) {  
  console.log(i);  
}
```

```
    i++;  
}
```

Output:

```
1  
2  
3
```

A missing update can produce an infinite loop:

```
let i = 1;  
  
while (i <= 3) {  
    console.log(i);  
}
```

i never changes.

4.4 do...while

```
let i = 5;  
  
do {  
    console.log(i);  
} while (i < 5);
```

Output:

```
5
```

The body executes before the first condition check.

while -> zero or more executions

do...while -> at least one execution

4.5 break

```
for (let i = 1; i <= 5; i++) {  
    if (i === 3) {  
        break;  
    }  
  
    console.log(i);  
}
```

Output:

```
1  
2
```

break ends the entire loop immediately.

Position matters:

```
for (let i = 1; i <= 5; i++) {  
    console.log(i);  
  
    if (i === 3) {  
        break;  
    }  
}
```

Output:

```
1  
2  
3
```

The value is printed before break executes.

4.6 continue

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue;  
  }  
  
  console.log(i);  
}
```

Output:

1
2
4
5

break -> end the whole loop

continue -> skip only the current iteration

4.7 Accumulator and counter patterns

Accumulator:

```
let sum = 0;  
  
for (let i = 1; i <= 4; i++) {  
  sum += i;  
}  
  
console.log(sum);
```

Output:

10

Counter:

```
let count = 0;  
  
for (let i = 1; i <= 6; i++) {  
  if (i % 2 === 0) {  
    count++;  
  }  
}  
  
console.log(count);
```

Output:

3

4.8 Combined trace pattern

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
  if (i === 2) {  
    continue;  
  }  
  
  if (i === 5) {  
    break;  
  }  
  
  sum += i;  
}
```



```
    }  
  
    sum += i;  
}  
  
console.log(sum);
```

Trace:

```
i = 1 -> add 1 -> sum = 1  
i = 2 -> continue -> sum = 1  
i = 3 -> add 3 -> sum = 4  
i = 4 -> add 4 -> sum = 8  
i = 5 -> break -> sum = 8
```

Output:

8

Assessment method: write the state after each iteration. Do not try to mentally jump to the final result.

5. Functions, Scope, Return, and Hoisting

5.1 Function declaration

```
function add(a, b) {  
    return a + b;  
}  
  
console.log(add(10, 20));
```

Output:

30

a and b are **parameters**. 10 and 20 are **arguments** supplied during the function call.

5.2 return has two jobs

```
function test() {  
    console.log("A");  
    return 10;  
    console.log("B");  
}  
  
console.log(test());
```

Output:

A
10

return:

1. sends a value back to the caller,
2. immediately ends that function call.

The second `console.log` is unreachable in normal execution.

5.3 console.log() is not return

```
function add(a, b) {  
    console.log(a + b);  
}
```

```
const result = add(2, 3);  
console.log(result);
```

Output:

```
5  
undefined
```

Why? The function prints 5, but it has no explicit **return**, so the returned value is **undefined**.

This distinction matters in callbacks. A callback that prints a value does not automatically return it.

5.4 Missing and extra arguments

```
function add(a, b) {  
    return a + b;  
}
```

```
console.log(add(10));
```

b becomes **undefined**.

10 + undefined -> NaN

Output:

```
NaN
```

Extra arguments can be supplied even when no matching parameters are declared:

```
function add(a, b) {  
    return a + b;  
}
```

```
console.log(add(10, 20, 30));
```

Output:

```
30
```

For this function, the third argument is not used.

5.5 Default parameters

```
function greet(name = "Guest") {  
    console.log("Hello " + name);  
}
```

```
greet();  
greet("Ravi");
```

Output:

```
Hello Guest  
Hello Ravi
```

A default value is useful when an argument may be omitted.

5.6 Local and global scope

```
let x = 100;
```

```
function test() {  
  let x = 20;  
  console.log(x);  
}
```

```
test();  
console.log(x);
```

Output:

```
20  
100
```

The local `x` shadows the global `x` inside the function. The global variable remains unchanged.

Why this matters in function questions

```
let x = 10;
```

```
function calculate(x) {  
  x = x + 5;  
  return x;  
}
```

```
const result = calculate(x);
```

```
console.log(result);  
console.log(x);
```

Output:

```
15  
10
```

The parameter `x` is a local variable for that call. Reassigning it does not reassign the outer primitive variable.

5.7 Function declaration hoisting

```
greet();
```

```
function greet() {  
  console.log("Hello");  
}
```

Output:

```
Hello
```

A function declaration can be called before its textual declaration in this common case.

Compare a function expression stored in `const`:

```
console.log(add(2, 3));
```

```
const add = function (a, b) {  
  return a + b;  
};
```

Result:

```
ReferenceError
```

Do not treat a `const` function expression like a function declaration.

5.8 Arrow functions

Normal function expression:

```
const add = function (a, b) {  
  return a + b;  
};
```

Arrow version:

```
const add = (a, b) => {  
  return a + b;  
};
```

Short implicit return:

```
const add = (a, b) => a + b;
```

Critical arrow return trap

```
const square = x => {  
  x * x;  
};
```

```
console.log(square(5));
```

Output:

undefined

Once a block body {} is used, an explicit `return` is needed:

```
const square = x => {  
  return x * x;  
};
```

This same trap appears inside `map()` callbacks.

5.9 Callback functions in event code

```
button.addEventListener("click", function () {  
  console.log("Clicked");  
});
```

The function is passed to the event system as a callback. It is not executed immediately.

Named callback:

```
function showMessage() {  
  console.log("Clicked");  
}
```

```
button.addEventListener("click", showMessage);
```

Correct:

`showMessage`

Potential mistake:

`showMessage()`

The parentheses call the function immediately and pass its returned value instead of passing the function itself.

6. Small Array Patterns Needed for Assessment Tasks

This section is intentionally compact. It covers only JavaScript-specific behavior already used in assessment-style questions and DOM processing.

6.1 push() return value

```
const nums = [1, 2, 3];
const result = nums.push(4);
```

```
console.log(result);
console.log(nums);
```

Output:

```
4
[1, 2, 3, 4]
```

push() mutates the array and returns the **new length**, not the updated array.

6.2 Reference mutation

```
const a = [10, 20, 30];
const b = a;
```

```
b.pop();
```

```
console.log(a);
console.log(b);
```

Output:

```
[10, 20]
[10, 20]
```

Both variables point to the same array object.

6.3 map() and the missing return trap

```
const nums = [1, 2, 3, 4];
```

```
const result = nums.map(num => {
  num * 2;
});
```

```
console.log(result);
```

Output:

```
[undefined, undefined, undefined, undefined]
```

Why? The callback block uses {}, so it does not implicitly return `num * 2`.

Correct:

```
const result = nums.map(num => {
  return num * 2;
});
```

or:

```
const result = nums.map(num => num * 2);
```

6.4 filter() plus map() chaining

```
const marks = [45, 80, 32, 90, 67];
```

```
const result = marks
  .filter(mark => mark >= 50)
  .map(mark => mark + 5);
```

```
console.log(result);
```

Execution:

```
filter -> [80, 90, 67]
```

```
map    -> [85, 95, 72]
```

Output:

```
[85, 95, 72]
```

Why this appears again in DOM work

`querySelectorAll()` returns a collection of elements. You may iterate through it and extract `.value` from each selected checkbox. The same idea of transforming multiple values appears again, even if the data source is DOM elements rather than a normal numeric array.

7. DOM Fundamentals

7.1 What is the DOM?

DOM stands for **Document Object Model**.

When the browser parses HTML, it exposes the page as a tree of objects that JavaScript can access and modify.

HTML source

- > browser parses it
- > DOM tree is created
- > JavaScript accesses DOM objects
- > DOM changes
- > browser updates the visible page

Example HTML:

```
<h2 id="welcome">Welcome User</h2>
```

JavaScript:

```
const heading = document.getElementById("welcome");  
heading.textContent = "Welcome Suyash";
```

The original file does not need to be rewritten. JavaScript modifies the in-memory DOM for the current page session. A refresh normally rebuilds the page from the original HTML unless data is stored somewhere persistent.

7.2 document

`document`

represents the current web document in browser JavaScript.

Therefore:

```
document.getElementById("welcome")
```

means: find the element with that ID in the current document.

8. Selecting Elements

8.1 getElementById()

HTML:

```
<h2 id="welcome">Welcome User</h2>
```

JavaScript:

```
const heading = document.getElementById("welcome");
```

Important syntax trap:

```
document.getElementById("welcome"); // correct
```

not:

```
document.getElementById("#welcome"); // wrong
```

getElementById expects the ID value, not CSS selector syntax.

8.2 querySelector()

querySelector() accepts CSS selector syntax and returns the **first matching element**.

```
document.querySelector("#hero");
document.querySelector(".card");
document.querySelector("h2");
document.querySelector("#profile-card h3");
```

Why #profile-card h3 is useful:

```
<article id="profile-card">
  
  <h3>Suyash Giri</h3>
</article>
```

You can select the heading without adding another ID:

```
const profileName = document.querySelector("#profile-card h3");
```

The CSS selector means “the first h3 inside #profile-card.”

8.3 querySelectorAll()

HTML:

```
<article class="card">React</article>
<article class="card">Spring Boot</article>
<article class="card">DSA</article>
```

JavaScript:

```
const cards = document.querySelectorAll(".card");
```

cards is a NodeList containing all matching elements.

```
console.log(cards.length);
```

Output:

3

Major trap

Wrong:

```
cards.textContent = "Updated";
```

cards is a collection, not a single element.

Correct:

```
cards.forEach(card => {
  card.textContent = "Updated";
});
```

Why `forEach` is used here

A separate DOM element must be modified on each iteration. `forEach` receives each element one at a time as `card`, so `card.textContent` is valid.

8.4 Selection decision table

Requirement	Use
One element with known ID	<code>getElementById("id")</code>
First match using any CSS selector	<code>querySelector("selector")</code>
All matches using a CSS selector	<code>querySelectorAll("selector")</code>

Assessment question:

Select only the first element with class `btn-primary`.

Answer:

```
const button = document.querySelector(".btn-primary");
```

Assessment question:

Select every course card.

Answer:

```
const cards = document.querySelectorAll(".card");
```

9. Reading and Modifying Page Content

9.1 `textContent`

HTML:

```
<p id="status">Pending</p>
```

Read:

```
const status = document.getElementById("status");  
console.log(status.textContent);
```

Output:

Pending

Change:

```
status.textContent = "Completed";
```

Use `textContent` for normal textual content.

9.2 `.value`

Inputs store user-entered data in `.value`.

```
<input type="text" id="full-name" value="Aman">  
  
const input = document.getElementById("full-name");  
console.log(input.value);
```

Output initially:

Aman

If the user edits the field to Ravi, then:


```
console.log(input.value);
```

prints:

Ravi

.value reads the current input value.

Assessment distinction

normal element text -> .textContent

input/select value -> .value

checkbox/radio state -> .checked

9.3 .checked

HTML:

```
<input type="checkbox" id="java-interest" checked>
```

JavaScript:

```
const checkbox = document.getElementById("java-interest");  
console.log(checkbox.checked);
```

Output:

true

After the user unchecks it:

false

9.4 .dataset

HTML:

```
<article id="profile-card" data-employee-id="101"></article>
```

JavaScript:

```
const profile = document.getElementById("profile-card");  
console.log(profile.dataset.employeeId);
```

Output:

101

Mapping rule:

data-employee-id -> dataset.employeeId

data-user-role -> dataset.userRole

Hyphenated data-* names become camelCase properties in dataset.

9.5 textContent versus innerHTML

```
message.textContent = "<strong>Hello</strong>";
```

The markup is treated as text.

```
message.innerHTML = "<strong>Hello</strong>";
```

The markup is parsed and rendered as HTML.

For simple user-entered text, prefer textContent. Inserting untrusted content with innerHTML can create security problems.

Assessment-level rule:

textContent -> plain text

innerHTML -> parse HTML markup

10. Classes, Styles, and Attributes

10.1 `classList.add()`

CSS:

```
.active {  
    background-color: lightblue;  
}
```

JavaScript:

```
card.classList.add("active");
```

The class is added to the element, and the CSS rule becomes applicable.

10.2 `classList.remove()`

```
card.classList.remove("active");
```

Use when the element must leave that visual/state category.

10.3 `classList.toggle()`

CSS:

```
.hidden {  
    display: none;  
}
```

JavaScript:

```
announcements.classList.toggle("hidden");
```

Behavior:

hidden absent -> add hidden

hidden present -> remove hidden

This is ideal for show/hide features.

10.4 `classList.contains()`

```
if (card.classList.contains("active")) {  
    console.log("Already active");  
}
```

Returns a Boolean.

10.5 Direct style modification

```
box.style.backgroundColor = "yellow";  
box.style.fontSize = "18px";
```

CSS names become camelCase in the `style` property:

background-color -> backgroundColor

font-size -> fontSize

margin-top -> marginTop

For multiple visual changes, prefer toggling a CSS class instead of putting all presentation logic into JavaScript.

10.6 Attributes

Read:

```
const src = image.getAttribute("src");
```

Set:

```
image.setAttribute("src", "new.jpg");
```

Remove:

```
input.removeAttribute("disabled");
```

Some attributes also have convenient DOM properties:

```
image.src = "new.jpg";  
input.disabled = true;  
input.disabled = false;
```

Assessment question pattern

Change the image source to `profile-new.jpg`.

Possible answer:

```
const image = document.getElementById("profile-img");  
image.setAttribute("src", "profile-new.jpg");
```

or:

```
image.src = "profile-new.jpg";
```

11. Events

11.1 What is an event?

An event represents something that happens in the browser.

Common events:

```
click -> button/link interaction  
input -> text input changes as the user types  
change -> checkbox/radio/select state changes  
submit -> form submission
```

11.2 addEventListener()

HTML:

```
<button id="help-btn" type="button">Help</button>
```

JavaScript:

```
const helpBtn = document.getElementById("help-btn");
```

```
helpBtn.addEventListener("click", function () {  
    console.log("Help button clicked");  
});
```

Execution:

```
script runs  
-> button is selected  
-> listener is registered  
-> callback waits  
-> user clicks  
-> callback executes
```

11.3 click

```
button.addEventListener("click", function () {  
    message.textContent = "Clicked";  
});
```

Use for explicit user actions such as Preview, Save, Delete, Toggle, or Select.

11.4 input

```
nameInput.addEventListener("input", function () {  
    console.log(nameInput.value);  
});
```

The handler can run as the user types. Useful for live preview or live validation.

11.5 change

```
checkbox.addEventListener("change", function () {  
    console.log(checkbox.checked);  
});
```

Useful for checkboxes, radio controls, and select elements when the selected state changes.

11.6 submit

```
form.addEventListener("submit", function (event) {  
    event.preventDefault();  
    console.log("Submitted");  
});
```

A form's normal browser behavior may submit/navigate/reload. `preventDefault()` lets JavaScript validate and decide what happens next.

11.7 event.target

```
button.addEventListener("click", function (event) {  
    console.log(event.target);  
});
```

`event.target` is the element where the event originated.

This becomes especially important when a parent listener handles events from many child buttons.

11.8 event.currentTarget

HTML:

```
<button id="save-btn">  
    <span>Save</span>  
</button>
```

```
button.addEventListener("click", function (event) {  
    console.log(event.target);  
    console.log(event.currentTarget);  
});
```

If the user clicks directly on the `<span>`:

```
event.target      -> span  
event.currentTarget -> button
```

`currentTarget` is the element whose listener is currently running.

12. Forms and Validation

12.1 The basic form flow

HTML:

```
<form id="profile-form">
  <input type="text" id="username">
  <button type="submit">Submit</button>
</form>
```

JavaScript:

```
const form = document.getElementById("profile-form");
const username = document.getElementById("username");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const value = username.value.trim();

  if (value === "") {
    console.log("Username required");
    return;
  }

  console.log("Valid form");
});
```

Why each line exists

```
const form = document.getElementById("profile-form");
```

The submit listener belongs on the form, because form submission is the event being handled.

```
const username = document.getElementById("username");
```

The input is selected because its current value must be read during validation.

```
form.addEventListener("submit", function (event) {
```

The validation should run when the user attempts to submit the form.

```
event.preventDefault();
```

The browser's default form submission is temporarily stopped so JavaScript can validate first.

```
const value = username.value.trim();
```

`.value` gets current user input. `.trim()` removes surrounding whitespace so a string containing only spaces becomes `""`.

```
if (value === "") {
```

The cleaned value is tested for emptiness.

```
return;
```

The callback stops immediately for the invalid case, preventing later success logic from running.

12.2 Why `.trim()` matters

```
nameInput.value === ""
```

fails to reject:

" "

Better:

```
nameInput.value.trim() === ""
```

Because:

```
" ".trim() -> ""
```

12.3 Checkbox validation

```
<input type="checkbox" id="terms">
const terms = document.getElementById("terms");

if (!terms.checked) {
  console.log("Please accept terms");
  return;
}
```

The Boolean `.checked` property is the correct source of checkbox state.

12.4 Multiple checked checkboxes

HTML:

```
<input type="checkbox" name="skills" value="Java" checked>
<input type="checkbox" name="skills" value="React">
<input type="checkbox" name="skills" value="DSA" checked>
```

Select only checked skill inputs:

```
const selectedSkills = document.querySelectorAll(
  'input[name="skills"]:checked'
);
```

Why this line works:

- `input` restricts the selector to input elements.
- `[name="skills"]` selects the inputs belonging to the skill group.
- `:checked` keeps only controls currently selected.
- `querySelectorAll` is used because multiple checkboxes can be checked simultaneously.

Extract values:

```
const values = [];

selectedSkills.forEach(skill => {
  values.push(skill.value);
});

console.log(values);
```

If Java and DSA are checked:

```
["Java", "DSA"]
```

12.5 Radio buttons

HTML:

```
<input type="radio" name="gender" value="Male">
<input type="radio" name="gender" value="Female">
```

Selected radio:

```
const gender = document.querySelector(
  'input[name="gender"]:checked'
);
```

querySelector is sufficient because one radio group normally has only one selected option.

Null trap

If nothing is selected:

```
const gender = document.querySelector(
  'input[name="gender"]:checked'
);
```

returns:

null

Wrong:

```
console.log(gender.value);
```

If gender is null, .value cannot be read.

Correct:

```
if (!gender) {
  console.log("Select gender");
  return;
}
```

```
console.log(gender.value);
```

12.6 Select element

HTML:

```
<select id="department">
  <option value="">Choose Department</option>
  <option value="engineering">Engineering</option>
  <option value="finance">Finance</option>
</select>
```

JavaScript:

```
const department = document.getElementById("department");
console.log(department.value);
```

If Engineering is selected:

engineering

Validation:

```
if (department.value === "") {
  console.log("Choose department");
  return;
}
```

12.7 Error messages in the page

HTML:

```
<p id="error-message"></p>
```

CSS:

```
.error {
  color: red;
  font-weight: bold;
}
```

JavaScript:

```
const errorMessage = document.getElementById("error-message");

errorMessage.textContent = "Name is required";
errorMessage.classList.add("error");
```

The same previously learned concepts are reused:

- `getElementById` selects the target.
 - `textContent` changes the message.
 - `classList.add` activates styling already defined in CSS.
-

12.8 Invalid input styling

CSS:

```
.invalid {
  border: 2px solid red;
}
```

JavaScript:

```
if (nameInput.value.trim() === "") {
  nameInput.classList.add("invalid");
} else {
  nameInput.classList.remove("invalid");
}
```

This is usually cleaner than writing border styles directly in JavaScript.

12.9 Early-return validation pattern

```
form.addEventListener("submit", function (event) {
  event.preventDefault();

  const name = nameInput.value.trim();
  const email = emailInput.value.trim();

  if (name === "") {
    console.log("Name required");
    return;
  }

  if (email === "") {
    console.log("Email required");
    return;
  }

  if (!terms.checked) {
    console.log("Accept terms");
    return;
  }

  console.log("Registration successful");
});
```

Why this is easy to reason about:

Each invalid condition exits immediately. The final success line is reached only if all earlier checks passed.

12.10 required, readonly, and disabled

HTML can provide built-in behavior:


```
<input id="name" required>
<input id="employee-id" value="101" readonly>
<input id="status" value="Active" disabled>
```

Important distinctions:

required -> browser can prevent empty submission

readonly -> user cannot edit, normally included in form submission

disabled -> control is disabled, normally not included in submitted form data

JavaScript can still read values from disabled/readonly elements if it has a DOM reference:

```
console.log(document.getElementById("status").value);
```

12.11 Button types inside forms

```
<button type="submit">Update Profile</button>
<button type="reset">Reset Form</button>
<button type="button">Preview Profile</button>
```

Use `type="button"` for a button that should perform JavaScript behavior without submitting the form.

A button inside a form without an explicit type can behave as a submit button, which is a common integration bug.

13. Creating, Inserting, and Removing Elements

13.1 createElement()

Existing HTML:

```
<ul id="skills-list">
  <li>Java</li>
</ul>
```

Create a new element:

```
const newSkill = document.createElement("li");
```

At this moment, the element exists in memory but is not yet visible in the page.

Add content:

```
newSkill.textContent = "React";
```

Insert it:

```
const skillsList = document.getElementById("skills-list");
skillsList.append(newSkill);
```

Final DOM conceptually:

```
<ul id="skills-list">
  <li>Java</li>
  <li>React</li>
</ul>
```

Why each concept reappears

- `getElementById` is needed again because `append` must be called on the parent list.
 - `textContent` is reused because the new `<li>` requires text.
 - `createElement` creates an element but does not decide where it belongs.
 - `append` performs the insertion into the DOM.
-

13.2 append() versus prepend()

```
list.append(item);
```

adds to the end.

```
list.prepend(item);
```

adds to the beginning.

13.3 Add-item assessment pattern

HTML:

```
<input id="skill-input">
<button id="add-btn" type="button">Add Skill</button>
<ul id="skills-list"></ul>
```

JavaScript:

```
const skillInput = document.getElementById("skill-input");
const addBtn = document.getElementById("add-btn");
const skillsList = document.getElementById("skills-list");
```

```
addBtn.addEventListener("click", function () {
  const skill = skillInput.value.trim();

  if (skill === "") {
    return;
  }

  const li = document.createElement("li");
  li.textContent = skill;
  skillsList.append(li);

  skillInput.value = "";
});
```

Execution map:

```
select input/button/list
-> wait for click
-> read current input value
-> trim and validate
-> create li
-> write textContent
-> append to list
-> clear input
```

This is a high-value integrated pattern because it combines multiple previously learned concepts in one short task.

13.4 remove()

```
const message = document.getElementById("message");
message.remove();
```

The element is removed from the DOM.

Difference:

```
message.classList.add("hidden");
```

keeps the element in the DOM but hides it through CSS.

```
message.remove();
```

removes the element itself.

13.5 Dynamic delete button

```
const li = document.createElement("li");
li.textContent = skill;

const deleteBtn = document.createElement("button");
deleteBtn.textContent = "Delete";

li.append(deleteBtn);
skillsList.append(li);

deleteBtn.addEventListener("click", function () {
  li.remove();
});
```

This is valid when the delete listener is attached at creation time. When many dynamic children exist, event delegation can be cleaner; that appears later.

14. DOM Traversal

Traversal means moving from one known DOM element to related elements.

14.1 parentElement

HTML:

```
<div class="card">
  <button class="complete-btn">Complete</button>
</div>

const button = document.querySelector(".complete-btn");
console.log(button.parentElement);
```

The direct parent is the `.card` element.

Trap

If the HTML changes:

```
<article class="card">
  <div>
    <button class="complete-btn">Complete</button>
  </div>
</article>
```

then `button.parentElement` is the inner `div`, not the `card`.

When the desired ancestor is identified by a selector, `closest()` is safer.

14.2 children

```
const card = document.querySelector(".card");
console.log(card.children);
```

`children` contains the direct child elements.

For a specifically named descendant, this is usually clearer:

```
const heading = card.querySelector("h3");
```

14.3 nextElementSibling and previousElementSibling

HTML:

```
<label>Name</label>
<input id="name">
<p class="error-message"></p>

const input = document.getElementById("name");

input.previousElementSibling; // label
input.nextElementSibling;    // p.error-message
```

A possible validation update:

```
input.nextElementSibling.textContent = "Name required";
```

Use this only when the markup relationship is stable and known.

14.4 closest()

HTML:

```
<article class="card">
  <div>
    <button class="delete-btn">Delete</button>
  </div>
</article>

const card = button.closest(".card");
```

closest() starts at the current element and searches upward until it finds the nearest matching element.

```
button
-> div
-> article.card [match]
```

Practical delete:

```
deleteBtn.addEventListener("click", function (event) {
  const card = event.target.closest(".card");
  card.remove();
});
```

event.target gives the clicked element. closest(".card") converts that clicked location into the card that should be removed.

15. Event Bubbling and Delegation

15.1 Event bubbling

HTML:

```
<div id="card">
  <button id="btn">Click</button>
</div>
```

JavaScript:

```
btn.addEventListener("click", function () {
  console.log("Button");
});

card.addEventListener("click", function () {
  console.log("Card");
});
```

A button click can produce:

Button

Card

The event originated at the child and bubbled upward to ancestors.

15.2 Event delegation

HTML:

```
<ul id="task-list">
  <li>Java <button class="delete-btn">Delete</button></li>
  <li>React <button class="delete-btn">Delete</button></li>
  <li>SQL <button class="delete-btn">Delete</button></li>
</ul>
```

Instead of attaching a separate listener to every delete button:

```
const taskList = document.getElementById("task-list");

taskList.addEventListener("click", function (event) {
  if (event.target.matches(".delete-btn")) {
    const item = event.target.closest("li");
    item.remove();
  }
});
```

Why each line exists

```
const taskList = document.getElementById("task-list");
```

The stable parent is selected because one listener will handle child events.

```
taskList.addEventListener("click", function (event) {
```

Clicks from child elements can bubble to this parent.

```
if (event.target.matches(".delete-btn")) {
```

Not every click inside the list should delete an item. `matches()` confirms that the actual clicked element is a delete button.

```
const item = event.target.closest("li");
```

The clicked button is converted into the list item that owns it.

```
item.remove();
```

Only that list item is removed.

Why delegation helps with dynamic elements

If a new `<li>` with a `.delete-btn` is appended later, the parent listener already exists. The new button can still be handled when its click bubbles to the parent.

15.3 `matches()`

```
if (event.target.matches(".select-btn")) {
  // selected target is a .select-btn
}
```

`matches()` returns whether the element satisfies a CSS selector.

15.4 preventDefault() versus stopPropagation()

preventDefault()

-> stop the browser's default action

-> example: prevent form submission/navigation

stopPropagation()

-> stop the event from propagating to ancestors

-> example: prevent a parent click handler from also running

Do not treat them as interchangeable.

16. NexaCorp Practical Reference Patterns

The examples below use the same element naming style as the NexaCorp training/recruitment portal. They are reference patterns, not a requirement to modify the project immediately.

16.1 Preview profile name

Assume:

```
<input type="text" id="full-name">
```

```
<article id="profile-card">
  <h3>Suyash Giri</h3>
</article>
```

```
<button type="button" id="preview-btn">Preview Profile</button>
```

JavaScript:

```
const nameInput = document.getElementById("full-name");
const previewBtn = document.getElementById("preview-btn");
const profileName = document.querySelector("#profile-card h3");

previewBtn.addEventListener("click", function () {
  const name = nameInput.value.trim();

  if (name === "") {
    console.log("Enter a valid name");
    return;
  }

  profileName.textContent = name;
});
```

Why previously learned concepts are reused

- `getElementById` selects known unique controls.
- `querySelector("#profile-card h3")` targets a descendant without adding another ID.
- `addEventListener("click", ...)` delays the logic until the user requests a preview.
- `.value` reads the current text field content.
- `.trim()` protects against whitespace-only names.
- `if` performs validation.
- `return` prevents invalid data from reaching the update line.
- `textContent` updates only the profile heading text.

Assessment prompt variation

On clicking Preview Profile, update the profile card heading only when the input is non-empty.
The same solution structure applies even if the IDs or visible text change.

16.2 Toggle announcements

Assume CSS:

```
.hidden {
  display: none;
}
```

HTML:

```
<aside id="announcements">...</aside>
<button id="help-btn" type="button">Help</button>
```

JavaScript:

```
const helpBtn = document.getElementById("help-btn");
const announcements = document.getElementById("announcements");
```

```
helpBtn.addEventListener("click", function () {
  announcements.classList.toggle("hidden");
});
```

Why `classList.toggle` rather than direct style manipulation?

The JavaScript owns the interaction state; CSS owns the visual rule. One line also supports both directions: hide and show.

16.3 Make only one course active

Assume:

```
<section id="courses">
  <article class="card">...</article>
  <article class="card active">...</article>
  <article class="card dsa-card">...</article>
</section>
```

Requirement:

Remove `active` from every card, then make the DSA card active.

```
const cards = document.querySelectorAll(".card");
```

```
cards.forEach(card => {
  card.classList.remove("active");
});
```

```
const dsaCard = document.querySelector(".dsa-card");
dsaCard.classList.add("active");
```

Why `querySelectorAll` first? The requirement applies to every existing card.

Why `forEach`? A `NodeList` is a collection. `classList` belongs to each individual element, not the collection itself.

Why `querySelector` for DSA? Only one target card is needed.

16.4 Read selected skills

Assume:

```
<input type="checkbox" name="skills" value="Java" checked>
<input type="checkbox" name="skills" value="React">
<input type="checkbox" name="skills" value="DSA" checked>

const selectedSkills = document.querySelectorAll(
  'input[name="skills"]:checked'
);

const skills = [];

selectedSkills.forEach(skill => {
  skills.push(skill.value);
});

console.log(skills);
```

Result:

```
["Java", "DSA"]
```

This is a common bridge between CSS-selector knowledge and JavaScript data processing.

16.5 Form submit validation

Assume:

```
<form id="profile-form">
  <input id="full-name">
  <input id="email" type="email">
  <button type="submit">Update Profile</button>
</form>

const form = document.getElementById("profile-form");
const nameInput = document.getElementById("full-name");
const emailInput = document.getElementById("email");

form.addEventListener("submit", function (event) {
  event.preventDefault();

  const name = nameInput.value.trim();
  const email = emailInput.value.trim();

  if (name === "") {
    console.log("Name required");
    return;
  }

  if (email === "") {
    console.log("Email required");
    return;
  }

  console.log("Profile valid");
});
```

Assessment reasoning:

1. The event belongs to the form, not only the submit button.
 2. `preventDefault` keeps the page available for validation.
 3. `.value` is used because these are inputs.
 4. `.trim()` catches whitespace-only input.
 5. Early returns make each failure case easy to trace.
-

17. Assessment Question Patterns and Answer Recipes

This section is the practical core of the notes. When a question appears, first identify its pattern.

Pattern 1: Exact output prediction

Prompt style

```
let x = 10;

function test(x) {
  x += 5;
  return x;
}

console.log(test(x));
console.log(x);
```

Answer method

1. Identify global state.
2. Identify local parameters.
3. Trace assignments in execution order.
4. Record each `console.log` exactly where it executes.

Answer:

```
15
10
```

Reason: the parameter `x` is local to the function call; the outer primitive variable is unchanged.

Pattern 2: Return-value trap

Prompt style

```
const nums = [1, 2, 3];
const result = nums.push(4);

console.log(result);
console.log(nums);
```

Answer

```
4
[1, 2, 3, 4]
```

Checklist:

- Did the method mutate the original value?
 - What did the method actually return?
-

Pattern 3: Missing return

Prompt style

```
const result = nums.map(num => {
  num * 2;
});
```

Diagnosis

The callback block does not return a value.

Minimum fix

```
const result = nums.map(num => {  
  return num * 2;  
});
```

or:

```
const result = nums.map(num => num * 2);
```

Pattern 4: Select one DOM element

Requirement

Select the element whose ID is `status`.

Answer:

```
const status = document.getElementById("status");
```

If the question explicitly asks for `querySelector`:

```
const status = document.querySelector("#status");
```

Pattern 5: Select all elements

Requirement

Select all elements with class `card`.

Answer:

```
const cards = document.querySelectorAll(".card");
```

If they must all change:

```
cards.forEach(card => {  
  card.classList.add("active");  
});
```

Do not call `cards.classList` on the `NodeList`.

Pattern 6: Read input and change text

Requirement

On button click, read the user's name and update the profile heading.

Recipe:

```
const input = document.getElementById("full-name");  
const button = document.getElementById("preview-btn");  
const heading = document.querySelector("#profile-card h3");
```

```
button.addEventListener("click", function () {  
  const name = input.value.trim();  
  
  if (name === "") {  
    return;  
  }  
  
  heading.textContent = name;  
});
```

Remember the chain:

select -> listen -> read -> validate -> modify

Pattern 7: Checkbox state

Requirement

Print whether the checkbox is selected.

```
const checkbox = document.getElementById("terms");
console.log(checkbox.checked);
```

Use `.checked`, not `.value`, to answer a yes/no state question.

Pattern 8: All selected checkboxes

```
const selected = document.querySelectorAll(
  'input[name="skills"]:checked'
);
```

Why `querySelectorAll`? Multiple checkboxes may be selected.

Pattern 9: Selected radio

```
const selected = document.querySelector(
  'input[name="gender"]:checked'
);
```

Before reading `.value`, guard against `null` if the HTML does not guarantee a selection.

Pattern 10: Hide/show using CSS

CSS:

```
.hidden {
  display: none;
}
```

JavaScript:

```
button.addEventListener("click", function () {
  panel.classList.toggle("hidden");
});
```

If the requirement is one-way hide only:

```
panel.classList.add("hidden");
```

Pattern 11: Prevent form submission and validate

```
form.addEventListener("submit", function (event) {
  event.preventDefault();

  if (nameInput.value.trim() === "") {
    console.log("Name required");
    return;
  }

  console.log("Valid");
});
```

Do not confuse `preventDefault()` with `stopPropagation()`.

Pattern 12: Create and append an element

Requirement:

Add the entered skill as a new list item.

```
const li = document.createElement("li");
li.textContent = skillInput.value.trim();
skillsList.append(li);
```

If validation is required, validate before creating/inserting.

Pattern 13: Remove the clicked card

```
container.addEventListener("click", function (event) {
  if (event.target.matches(".delete-btn")) {
    const card = event.target.closest(".card");
    card.remove();
  }
});
```

This pattern combines event delegation with traversal.

Pattern 14: Change an attribute

Requirement:

Change the image source after a button click.

```
button.addEventListener("click", function () {
  image.setAttribute("src", "new-image.jpg");
});
```

or:

```
image.src = "new-image.jpg";
```

Pattern 15: Bug caused by HTML button type

If a JavaScript-only button is inside a form and clicking it unexpectedly submits the form, inspect the HTML:

```
<button>Preview</button>
```

Safer explicit version:

```
<button type="button">Preview</button>
```

18. High-Frequency Traps

Trap	Wrong assumption	Correct rule
<code>getElementById("#x")</code>	IDs use # everywhere	<code>getElementById("x")</code> does not use #
<code>querySelectorAll()</code>	returns one element	returns a NodeList of all matches
<code>nodes.textContent</code>	NodeList behaves like an element	iterate over elements
input text	use <code>textContent</code>	use <code>.value</code>

Trap	Wrong assumption	Correct rule
checkbox state	use <code>.value</code>	use <code>.checked</code>
<code>querySelector</code> no match	returns empty collection	returns <code>null</code>
<code>querySelectorAll</code> no match	returns <code>null</code>	returns an empty <code>NodeList</code>
radio <code>.value</code>	always safe	selected radio may be <code>null</code>
<code>return</code>	only returns data	also stops the current function call
arrow <code>{}</code>	implicit return still happens	block body needs explicit <code>return</code>
<code>console.log</code>	same as <code>return</code>	printing and returning are different
<code>push()</code>	returns array	returns new length
array alias	<code>b = a</code> copies array	both reference same array
<code>break</code>	skips one iteration	ends entire loop
<code>continue</code>	ends loop	skips current iteration
<code>==</code>	same as <code>===</code>	coercion can occur with <code>==</code>
<code>"0"</code>	falsy because it looks like zero	non-empty string is truthy
<code>typeof null</code>	<code>null</code>	returns <code>"object"</code>
<code>typeof NaN</code>	special NaN type	returns <code>"number"</code>
<code>event.target</code>	always listener element	actual event-origin element
<code>event.currentTarget</code>	event-origin element	element whose handler is running
<code>preventDefault</code>	stops bubbling	stops default browser action
<code>stopPropagation</code>	stops form submission	stops propagation toward ancestors
<code>parentElement</code>	always desired card	only direct parent; use <code>closest()</code> when needed
<code>classList.toggle</code>	only adds	adds if absent, removes if present
<code>innerHTML</code>	same as <code>textContent</code>	parses markup
script timing	element lookup always works	lookup can return <code>null</code> before DOM is ready

19. Mixed Assessment Practice with Answers

Use this section for later revision. Try to answer each problem before reading the solution.

Q1 - Scope and function parameter

```
let x = 10;

function test(x) {
  x += 5;
  return x;
}

console.log(test(x));
console.log(x);
```

Answer

15
10

The function parameter is local to the call.

Q2 - Independent conditions

```
let score = 85;
```

```
if (score >= 50) {  
    console.log("Pass");  
}  
  
if (score >= 80) {  
    console.log("Excellent");  
} else {  
    console.log("Average");  
}
```

Answer

Pass
Excellent

The two if statements are independent. The `else` belongs only to the second if.

Q3 - Function control flow

```
function check(num) {  
    if (num % 2 === 0) {  
        return "Even";  
    }  
  
    console.log("Checking");  
    return "Odd";  
}  
  
console.log(check(6));  
console.log(check(5));
```

Answer

Even
Checking
Odd

For 6, return "Even" stops the function before Checking. For 5, the first branch is skipped, so Checking prints before Odd is returned and printed.

Q4 - DOM selector fill-in

HTML:

```
<h2 id="welcome">Welcome User</h2>
```

Complete:

```
const heading = _____;  
heading._____ = "Welcome Suyash";
```

Answer

```
const heading = document.getElementById("welcome");  
heading.textContent = "Welcome Suyash";
```

Q5 - Current input value

HTML initially:

```
<input id="full-name" value="Aman">
```

The user changes the visible input to Ravi and then this runs:

```
console.log(document.getElementById("full-name").value);
```

Answer

Ravi

.value reads the current value.

Q6 - NodeList bug

Wrong:

```
const cards = document.querySelectorAll(".card");
cards.classList.add("active");
```

Fix

```
const cards = document.querySelectorAll(".card");
```

```
cards.forEach(card => {
  card.classList.add("active");
});
```

classList belongs to each element, not the NodeList itself.

Q7 - Toggle visibility

CSS:

```
.hidden {
  display: none;
}
```

Complete:

```
helpBtn.addEventListener("click", function () {
  announcements. _____;
});
```

Answer

```
announcements.classList.toggle("hidden");
```

Q8 - Form submit

Why is this line present?

```
event.preventDefault();
```

Answer

It prevents the browser's normal default form submission/navigation so JavaScript can validate or update the UI first.

Q9 - Whitespace validation

Complete the condition so "" and " " are both invalid:

```
if ( _____ ) {
  console.log("Name required");
}
```

Answer

```
if (nameInput.value.trim() === "") {  
    console.log("Name required");  
}
```

Q10 - Selected checkboxes

Complete:

```
const selectedSkills = document.querySelectorAll(  
    '-----'  
);
```

Answer

```
const selectedSkills = document.querySelectorAll(  
    'input[name="skills"]:checked'  
);
```

Q11 - Radio null trap

```
const gender = document.querySelector(  
    'input[name="gender"]:checked'  
);
```

```
console.log(gender.value);
```

What can fail?

Answer

If no radio is checked, `gender` is `null`, so reading `gender.value` fails.

Safer:

```
if (!gender) {  
    console.log("Select gender");  
    return;  
}
```

```
console.log(gender.value);
```

Q12 - Dynamic list item

Requirement: add the current input value as a new `<li>`.

Answer pattern

```
const value = input.value.trim();
```

```
if (value === "") {  
    return;  
}
```

```
const li = document.createElement("li");  
li.textContent = value;  
list.append(li);
```

Q13 - Delete clicked card using delegation

Answer pattern

```
container.addEventListener("click", function (event) {  
    if (event.target.matches(".delete-btn")) {  
        const card = event.target.closest(".card");  
        card.remove();  
    }  
});
```

Q14 - target versus currentTarget

HTML:

```
<button id="save-btn"><span>Save</span></button>
```

If the listener is on the button and the user clicks the span:

Answer

```
event.target      -> span  
event.currentTarget -> button
```

Q15 - preventDefault versus propagation

Answer

```
preventDefault()  
-> cancel the browser's default action
```

```
stopPropagation()  
-> stop the event from continuing through ancestors
```

Q16 - Full integrated implementation

Requirement:

When Preview Profile is clicked, read the current full name. If it is empty or whitespace only, print **Enter a valid name** and do not update the profile. Otherwise update the `<h3>` inside `#profile-card`.

Answer

```
const nameInput = document.getElementById("full-name");  
const previewBtn = document.getElementById("preview-btn");  
const profileName = document.querySelector("#profile-card h3");  
  
previewBtn.addEventListener("click", function () {  
    const name = nameInput.value.trim();  
  
    if (name === "") {  
        console.log("Enter a valid name");  
        return;  
    }  
  
    profileName.textContent = name;  
});
```

Why every line exists

- `nameInput`: required to read current user input.
 - `previewBtn`: required because the feature starts on a click.
 - `profileName`: required because only the profile heading must change.
 - `addEventListener`: connects user action to JavaScript.
 - `.value`: gets what the user currently typed.
 - `.trim()`: rejects whitespace-only input.
 - `if`: chooses invalid versus valid flow.
 - `return`: ensures invalid input cannot reach the update.
 - `.textContent`: updates the visible name safely as text.
-

20. Last-Minute Revision Sheet

Core JavaScript

```
let    -> block scope, can reassign
const -> block scope, cannot reassign binding
var    -> function scope

typeof null -> "object"
typeof NaN  -> "number"

0    -> falsy
"0"  -> truthy
""   -> falsy

==  -> coercive equality
=== -> strict equality

! > && > ||

break    -> end loop
continue -> skip current iteration

return -> send value + stop function
no explicit return -> undefined

x => x * 2          -> implicit return
x => { x * 2; }     -> undefined
x => { return x * 2; } -> correct explicit return
```

DOM selection

```
document.getElementById("id")
document.querySelector(".class")
document.querySelectorAll(".class")

getElementById -> one ID, no #
querySelector  -> first CSS-selector match
querySelectorAll -> all matches / NodeList
```

DOM values

```
normal text    -> element.textContent
input/select   -> element.value
checkbox/radio    -> element.checked
data-user-id   -> element.dataset.userId
```

Events

```
element.addEventListener("click", function (event) {  
    // behavior  
});  
  
click -> click action  
input -> live input changes  
change -> selection/state change  
submit -> form submission  
  
event.target -> actual event origin  
event.currentTarget -> element whose listener is running
```

Forms

```
form.addEventListener("submit", function (event) {  
    event.preventDefault();  
});  
  
if (nameInput.value.trim() === "") {  
    return;  
}  
  
const selectedRadio = document.querySelector(  
    'input[name="gender"]:checked'  
);  
  
const selectedCheckboxes = document.querySelectorAll(  
    'input[name="skills"]:checked'  
);
```

Classes

```
element.classList.add("active");  
element.classList.remove("active");  
element.classList.toggle("hidden");  
element.classList.contains("active");
```

Attributes

```
element.getAttribute("src");  
element.setAttribute("src", "new.jpg");  
element.removeAttribute("disabled");
```

Dynamic DOM

```
const li = document.createElement("li");  
li.textContent = "React";  
list.append(li);  
li.remove();
```

Traversal

```
element.parentElement  
element.children  
element.nextElementSibling
```

```
element.previousElementSibling
element.closest(".card")
```

Delegation pattern

```
parent.addEventListener("click", function (event) {
  if (event.target.matches(".delete-btn")) {
    const item = event.target.closest(".item");
    item.remove();
  }
});
```

Universal assessment workflow

When a frontend task looks unfamiliar, translate the requirement into these questions:

1. What element(s) do I need? -> SELECT
2. When should behavior happen? -> EVENT
3. What current data do I need? -> READ
4. What conditions must be checked? -> VALIDATE
5. What must change? -> MODIFY

Example:

On clicking Save, if the name is non-empty, set the profile heading and add the **active** class.

Translation:

```
SELECT -> save button, input, heading/profile
EVENT  -> click
READ   -> input.value
VALIDATE -> trim() !== ""
MODIFY -> textContent + classList.add
```

That translation step is often more important than memorizing a large number of methods.

21. Coverage Checklist

Use this list after a long gap. If every line feels familiar and you can write a small example without looking, the covered JavaScript portion is ready for a mixed assessment.

Core language

- ☐ `var, let, const`
- ☐ Block scope versus function scope
- ☐ `typeof null, typeof NaN`
- ☐ Numeric/string coercion with `+` and arithmetic
- ☐ `==, ===, !=, !==`
- ☐ Truthy/falsy
- ☐ `!, &&, ||`
- ☐ Short-circuit behavior
- ☐ Ternary operator
- ☐ `for, while, do...while`
- ☐ `break, continue`
- ☐ Accumulator/counter tracing

Functions

- ☐ Function declaration

- ☐ Parameters versus arguments
- ☐ `return` versus `console.log`
- ☐ Missing arguments and `undefined`
- ☐ Default parameters
- ☐ Local/global scope and shadowing
- ☐ Function declaration hoisting
- ☐ Function expression
- ☐ Arrow function
- ☐ Explicit return when `{}` is used
- ☐ Callback reference versus immediate function call

Small JavaScript array support

- ☐ `push()` returns new length
- ☐ Shared array reference mutation
- ☐ `map()` callback return behavior
- ☐ `filter()` + `map()` chaining
- ☐ `forEach()` for DOM collections

DOM

- ☐ DOM definition
- ☐ `document`
- ☐ `getElementById`
- ☐ `querySelector`
- ☐ `querySelectorAll`
- ☐ `NodeList` versus one DOM element
- ☐ `textContent`
- ☐ `value`
- ☐ `checked`
- ☐ `dataset`
- ☐ `innerHTML` basic difference

CSS control from JavaScript

- ☐ `classList.add`
- ☐ `classList.remove`
- ☐ `classList.toggle`
- ☐ `classList.contains`
- ☐ `element.style` camelCase properties
- ☐ `getAttribute`
- ☐ `setAttribute`
- ☐ `removeAttribute`

Events

- ☐ `addEventListener`
- ☐ `click`
- ☐ `input`
- ☐ `change`
- ☐ `submit`
- ☐ `event.target`
- ☐ `event.currentTarget`
- ☐ Callback execution timing

Forms

- ☐ `preventDefault`
- ☐ `.trim()` validation
- ☐ Early return validation
- ☐ Checkbox validation

- ☐ Selected checkboxes selector
- ☐ Selected radio selector and null guard
- ☐ `select.value`
- ☐ Error message with `textContent`
- ☐ Invalid-state CSS class
- ☐ `required`, `readonly`, `disabled`
- ☐ `submit`, `reset`, and `button` button types

Dynamic DOM

- ☐ `createElement`
- ☐ `append`
- ☐ `prepend`
- ☐ `remove`
- ☐ Add-item flow
- ☐ Dynamic delete flow

Traversal and delegation

- ☐ `parentElement`
 - ☐ `children`
 - ☐ `nextElementSibling`
 - ☐ `previousElementSibling`
 - ☐ `closest`
 - ☐ Event bubbling
 - ☐ Event delegation
 - ☐ `matches`
 - ☐ `stopPropagation` versus `preventDefault`
-

Final Recall Map

If only one page of these notes can be recalled, remember this:

HTML gives JavaScript elements to work with.

CSS gives JavaScript classes/states to switch.

JavaScript connects user actions to DOM changes.

SELECT

-> `getElementById` / `querySelector` / `querySelectorAll`

LISTEN

-> `addEventListener`

READ

-> `textContent` / `value` / `checked` / `dataset`

VALIDATE

-> `trim` / `if` / logical operators / `return`

MODIFY

-> `textContent` / `classList` / `attributes` / `createElement` / `append` / `remove`

For an unfamiliar assessment prompt, do not search memory for an exact previously solved question. Convert the prompt into this execution flow and build the answer from the requirement.

End of Notes